Aula 15:
Computação Evolutiva e Conexionista – RNA's: Formas de Aprendizado

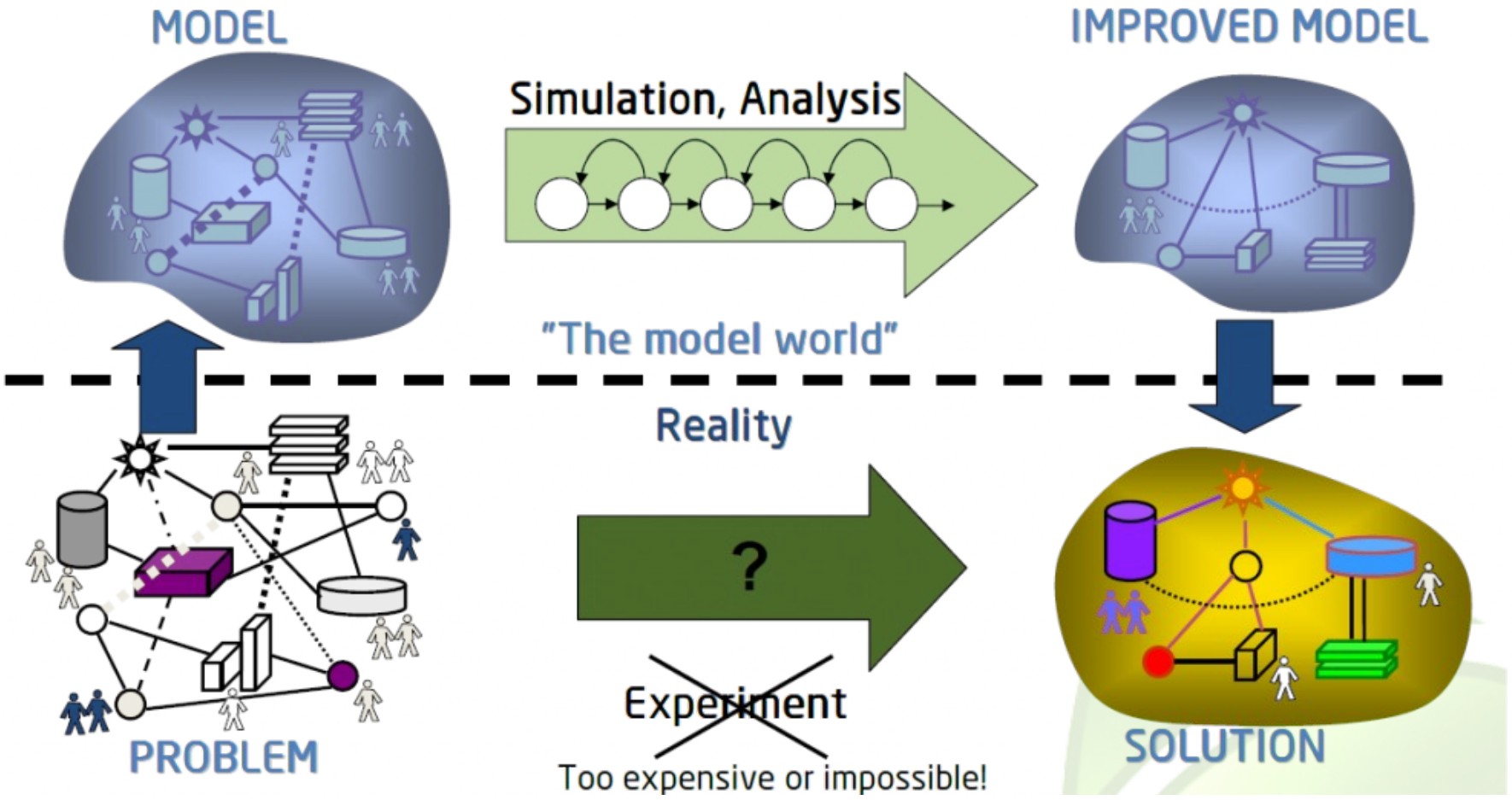
Prof. Hugo Puertas de Araújo
hugo.puertas@ufabc.edu.br
Sala: 509.2 (5º andar / Torre 2)

Agenda

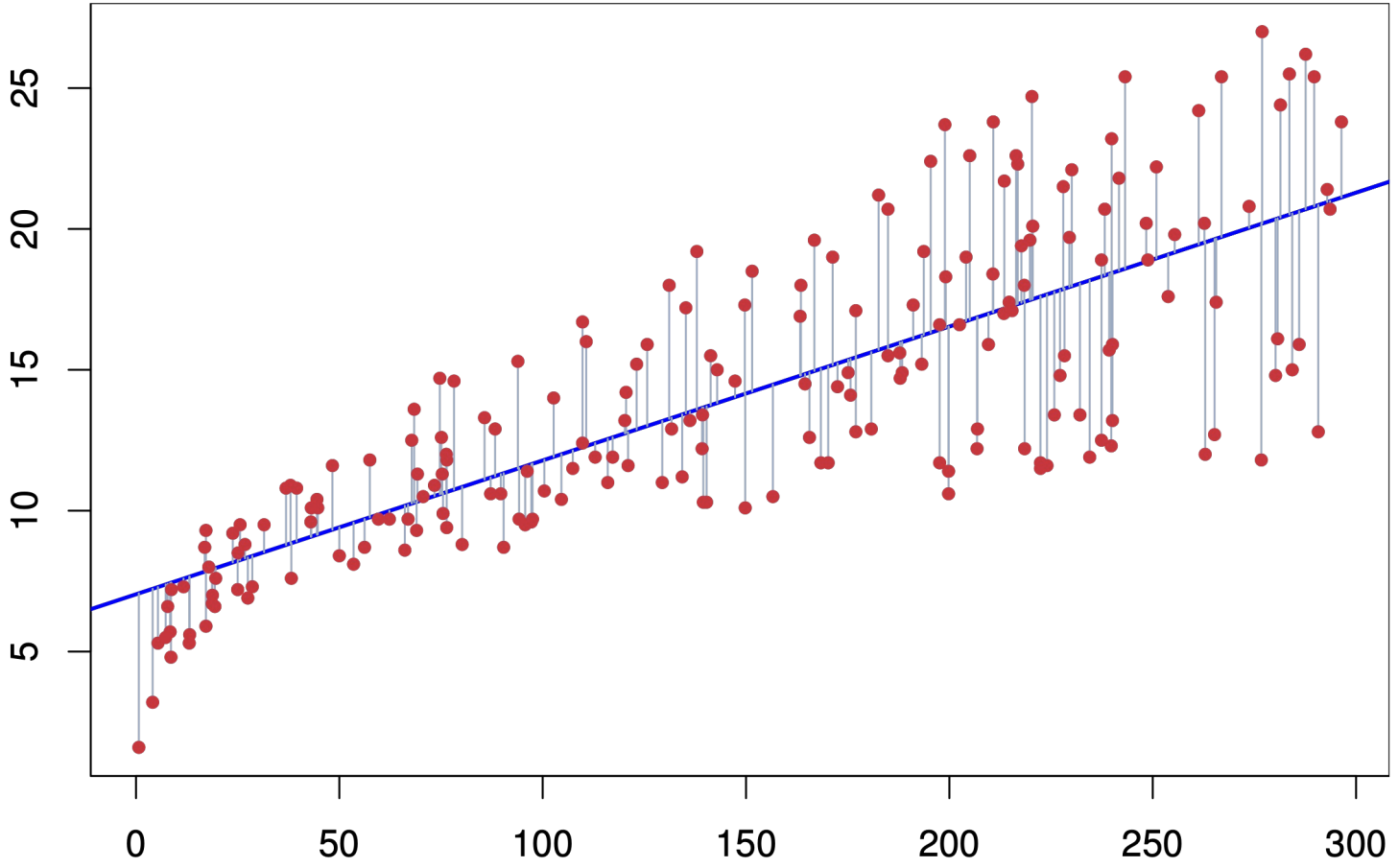
- Modelos
- Formas de aprendizado
  - Supervisionado
  - Não-supervisionado
  - Aprendizado por Reforço
- Solução de problemas
- Treinamento das RNA's

Computação Evolutiva e Conexionista

Modelos em Machine Learning

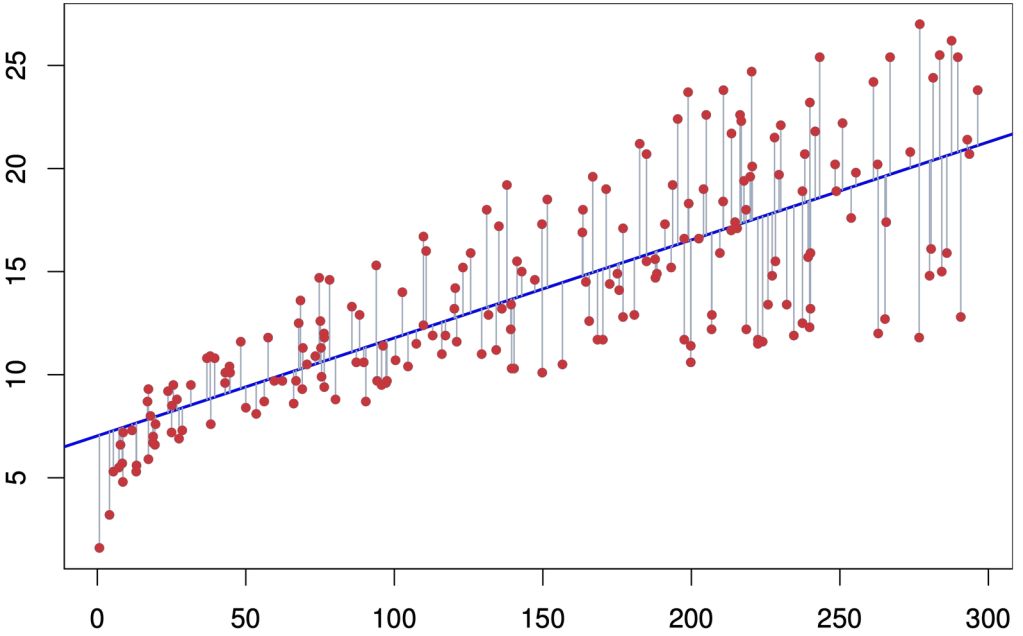


Caso simples: Regressão linear



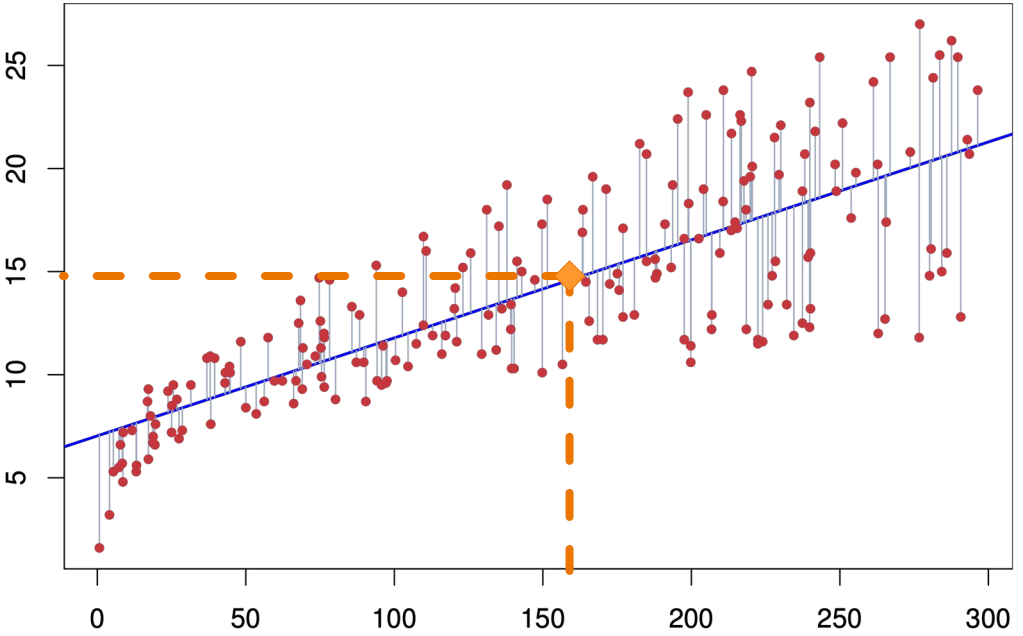
Caso simples: Regressão linear

- Obtenção dos dados (medidas, tabela);
- Seleção do modelo: linear;
- Ajuste dos parâmetros do modelo: treinamento
- Uso do modelo: inferência

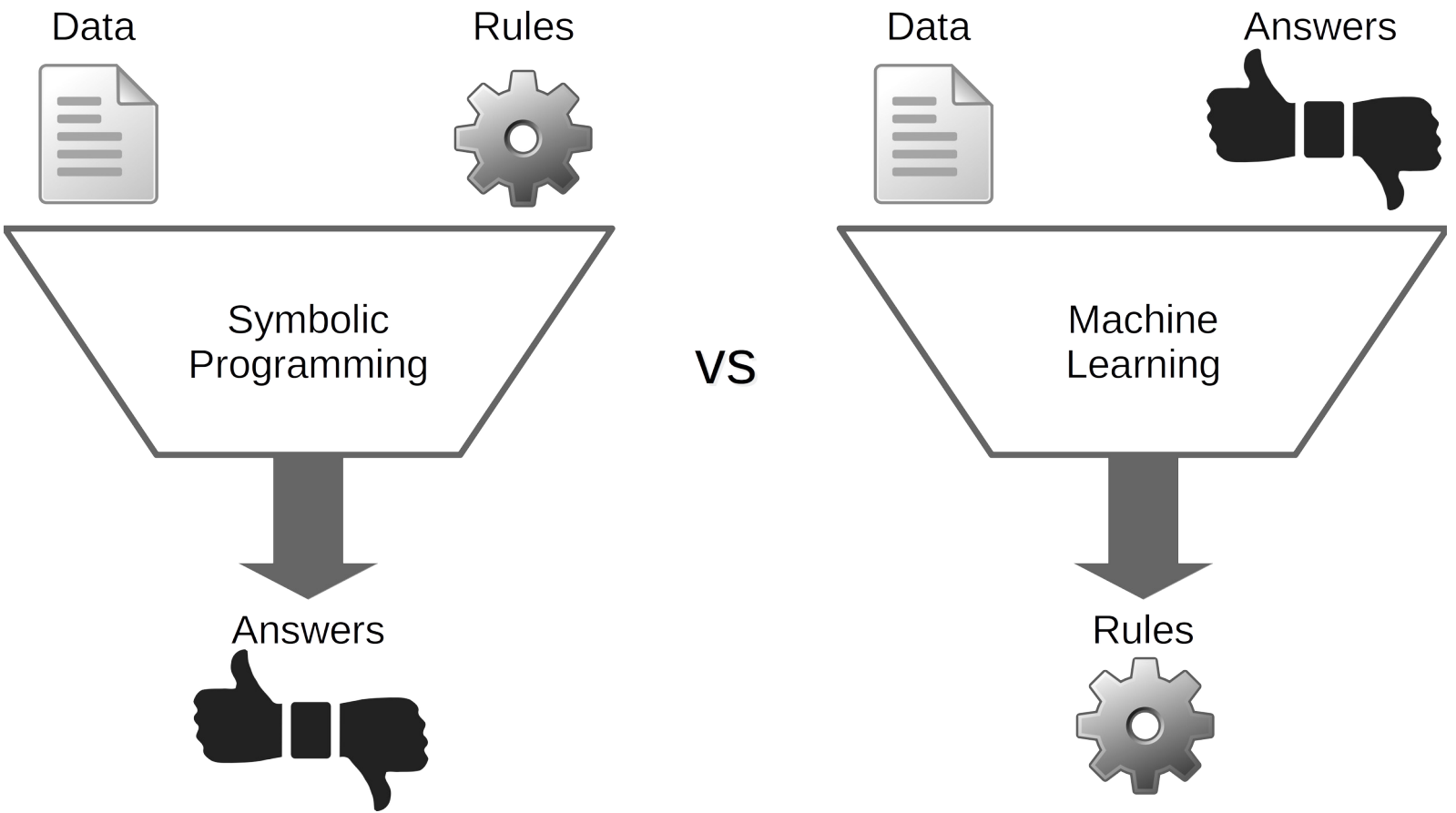


Caso simples: Regressão linear

- Obtenção dos dados (medidas, tabela);
- Seleção do modelo: linear;
- Ajuste dos parâmetros do modelo: treinamento
- Uso do modelo: inferência

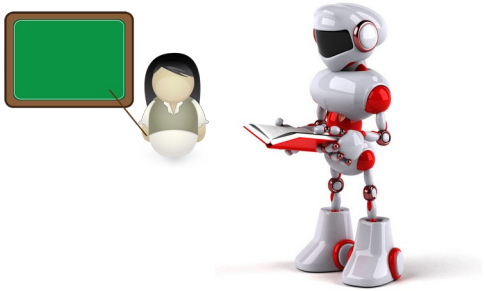


Modelos em Machine Learning

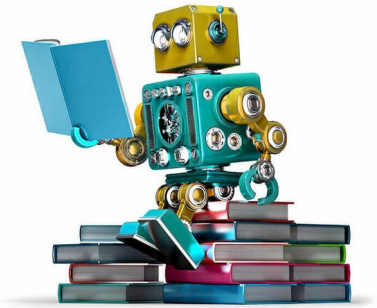


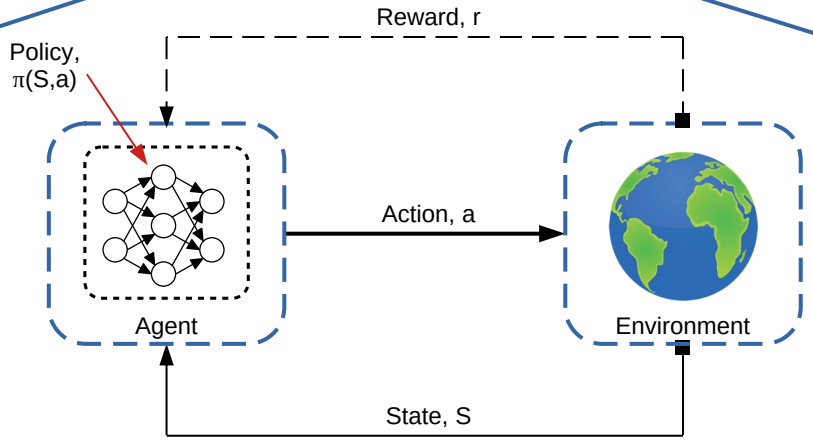
Formas de aprendizado

Supervisionado



Não Supervisionado

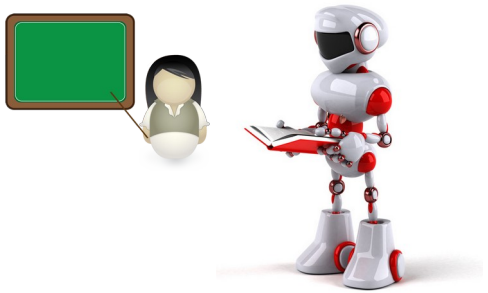




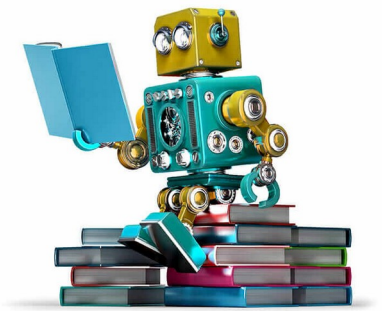
Por Reforço

Formas de aprendizado

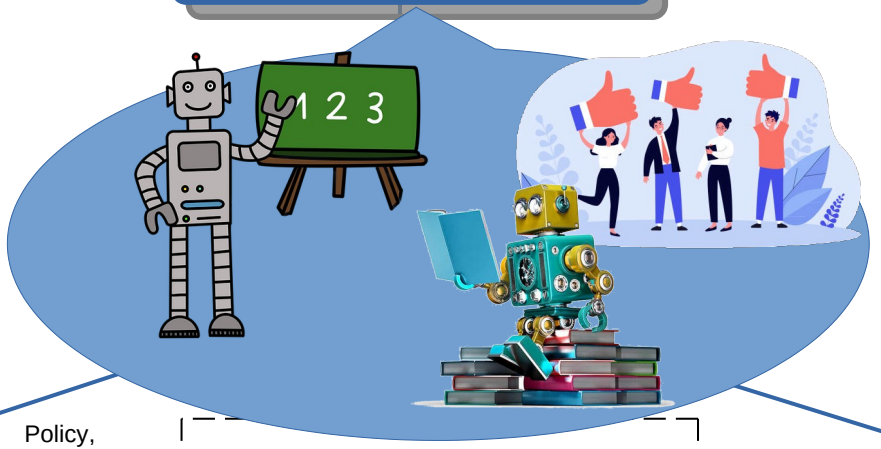
Supervisionado

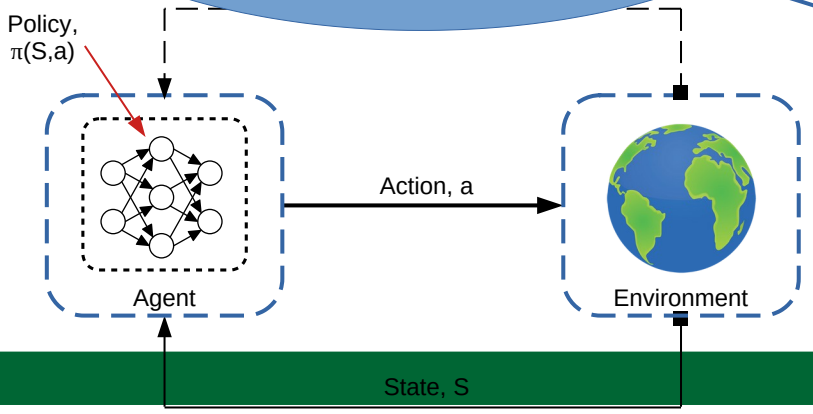


Não Supervisionado



Semi Supervisionado

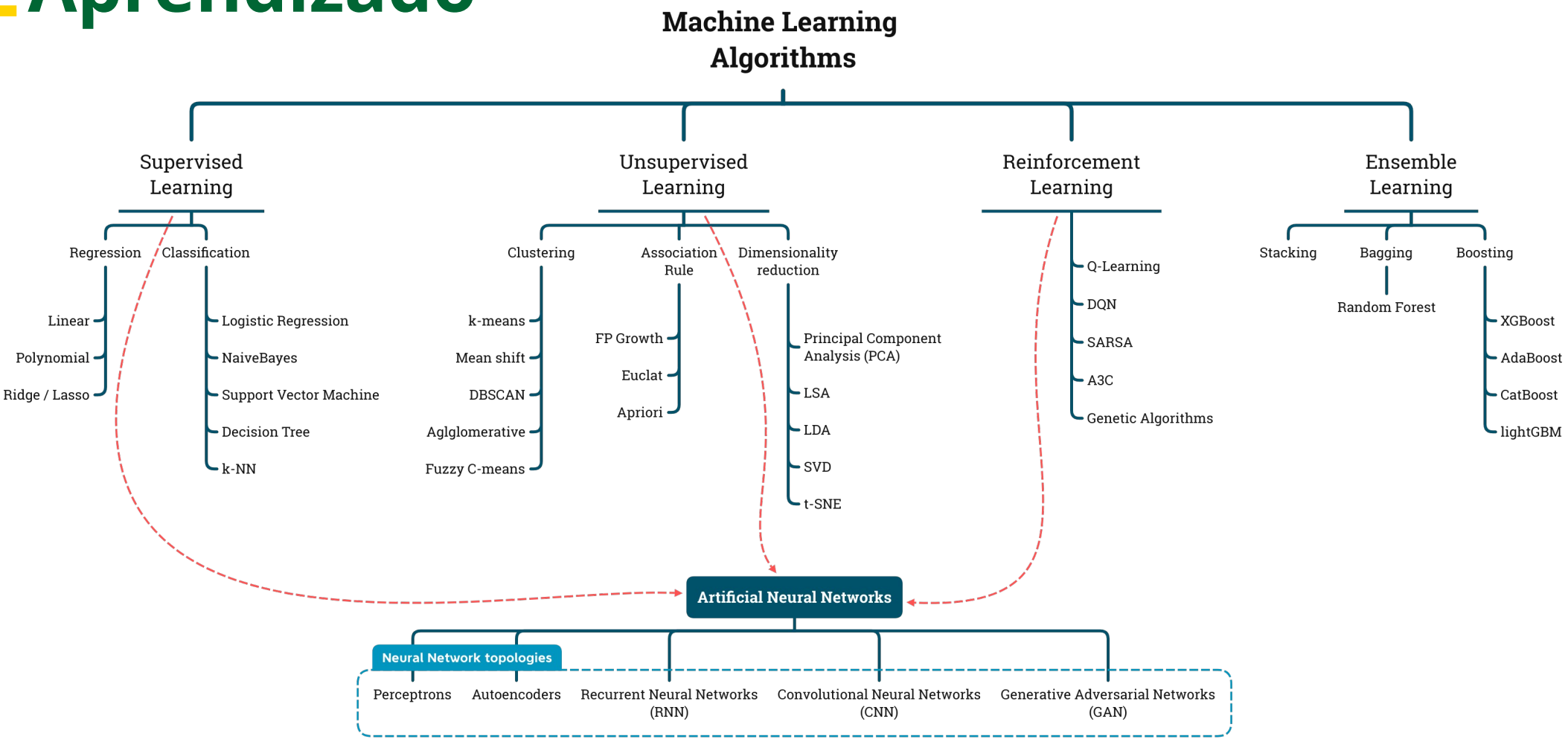




Por Reforço

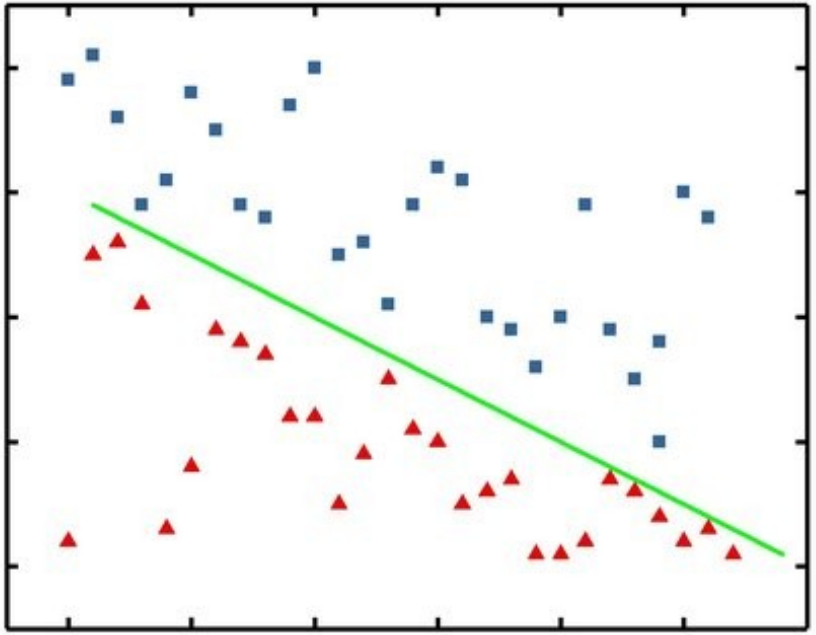
CMCC
Centro de Matemática, Computação e Cognição

Aprendizado

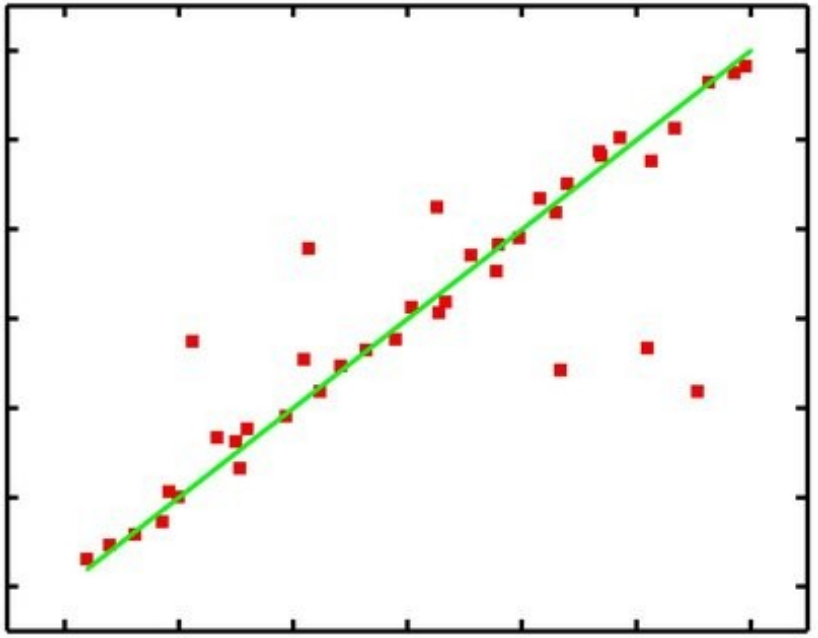


Formas de aprendizado

Supervisionado



(a) Logistic Regression



(b) Linear Regression

Classificação

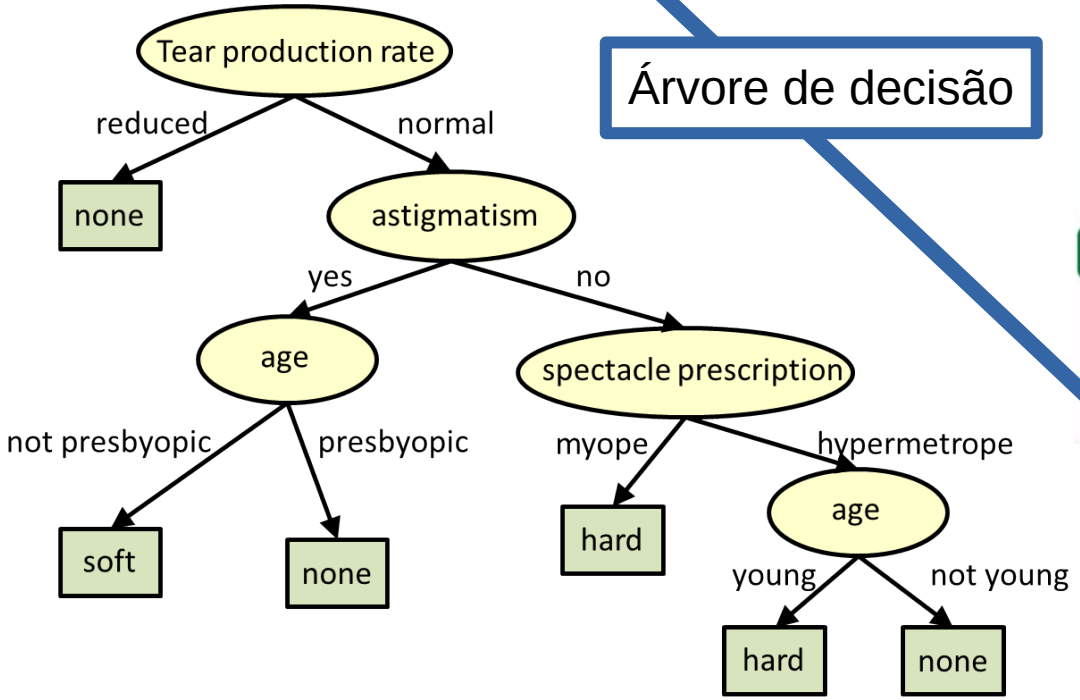
Regressão

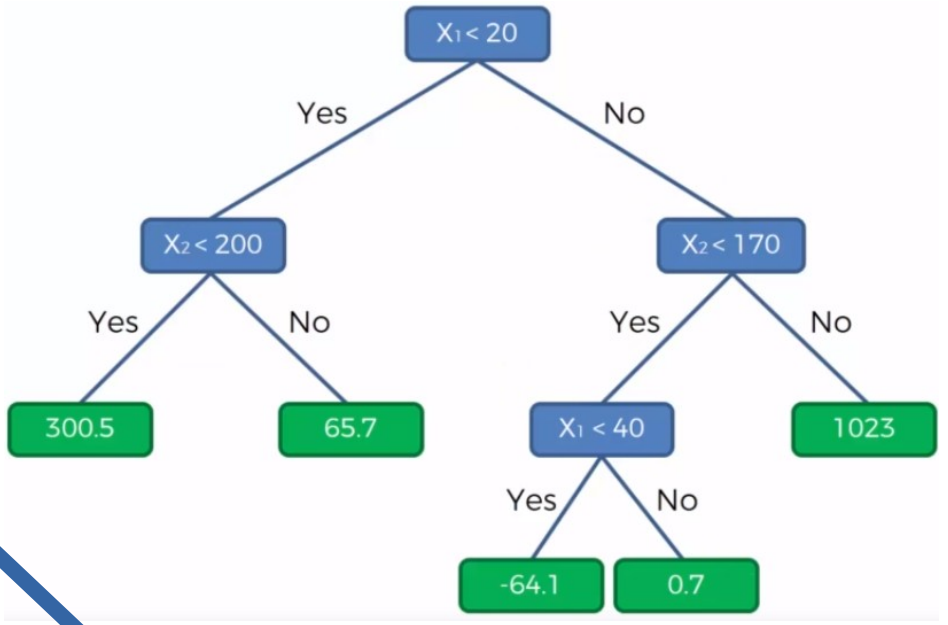
CMCC
Centro de Matemática, Computação e Cognição

Formas de aprendizado

Supervisionado

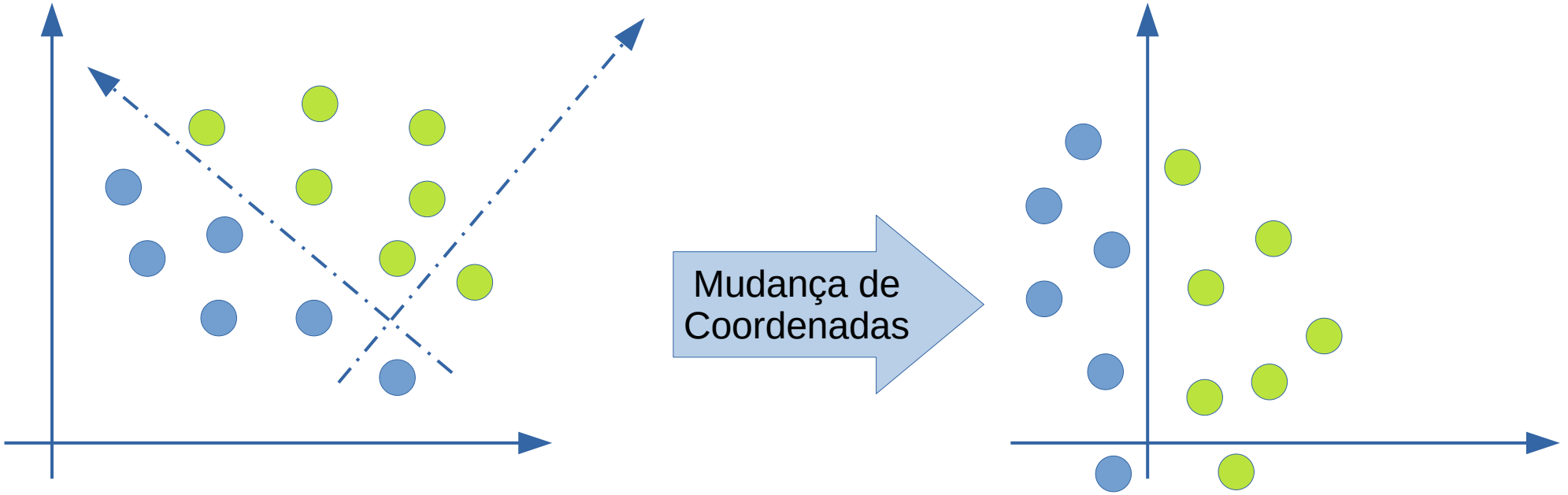
Árvore de decisão





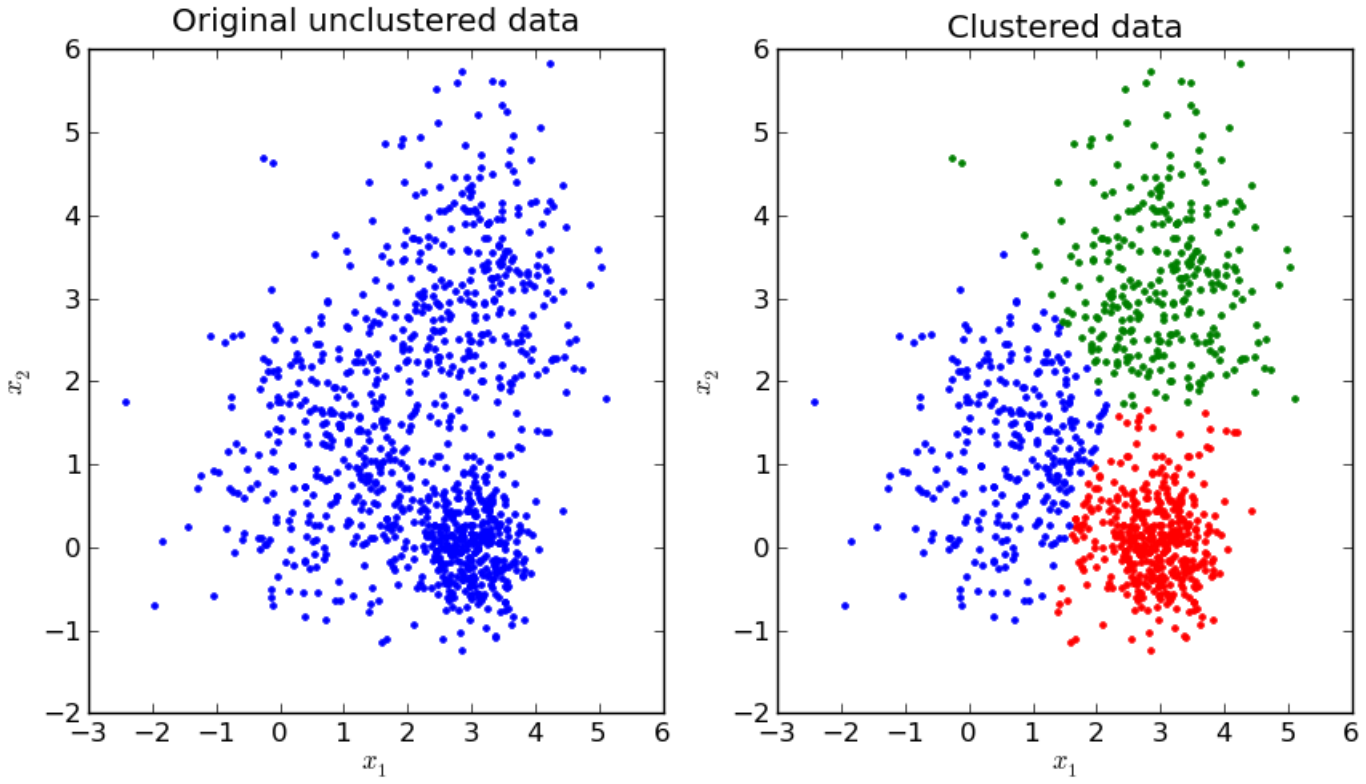
Formas de aprendizado

Não Supervisionado



Formas de aprendizado

Não Supervisionado

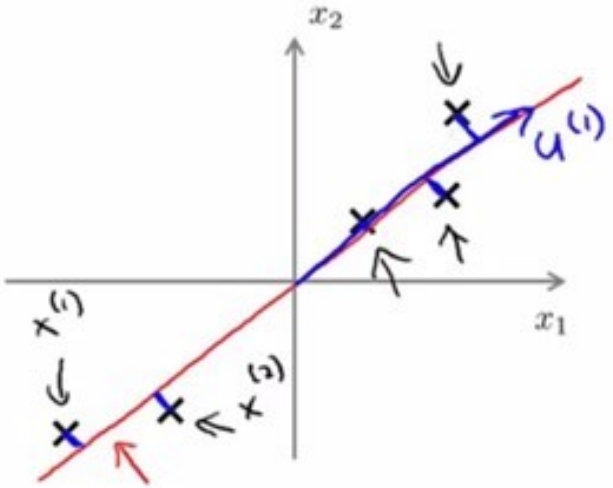


Classificação x Agrupamento

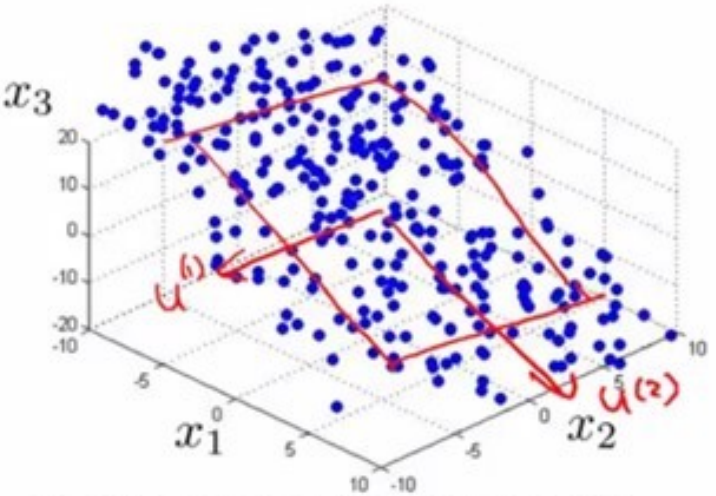
- Classificação
  - Define fronteiras entre os grupos de amostras
  - Supervisionado
- Agrupamento (Cluster)
  - Define centros dos grupos de amostras
  - Não-supervisionado

Redução de dimensionalidade

Principal Component Analysis (PCA) algorithm



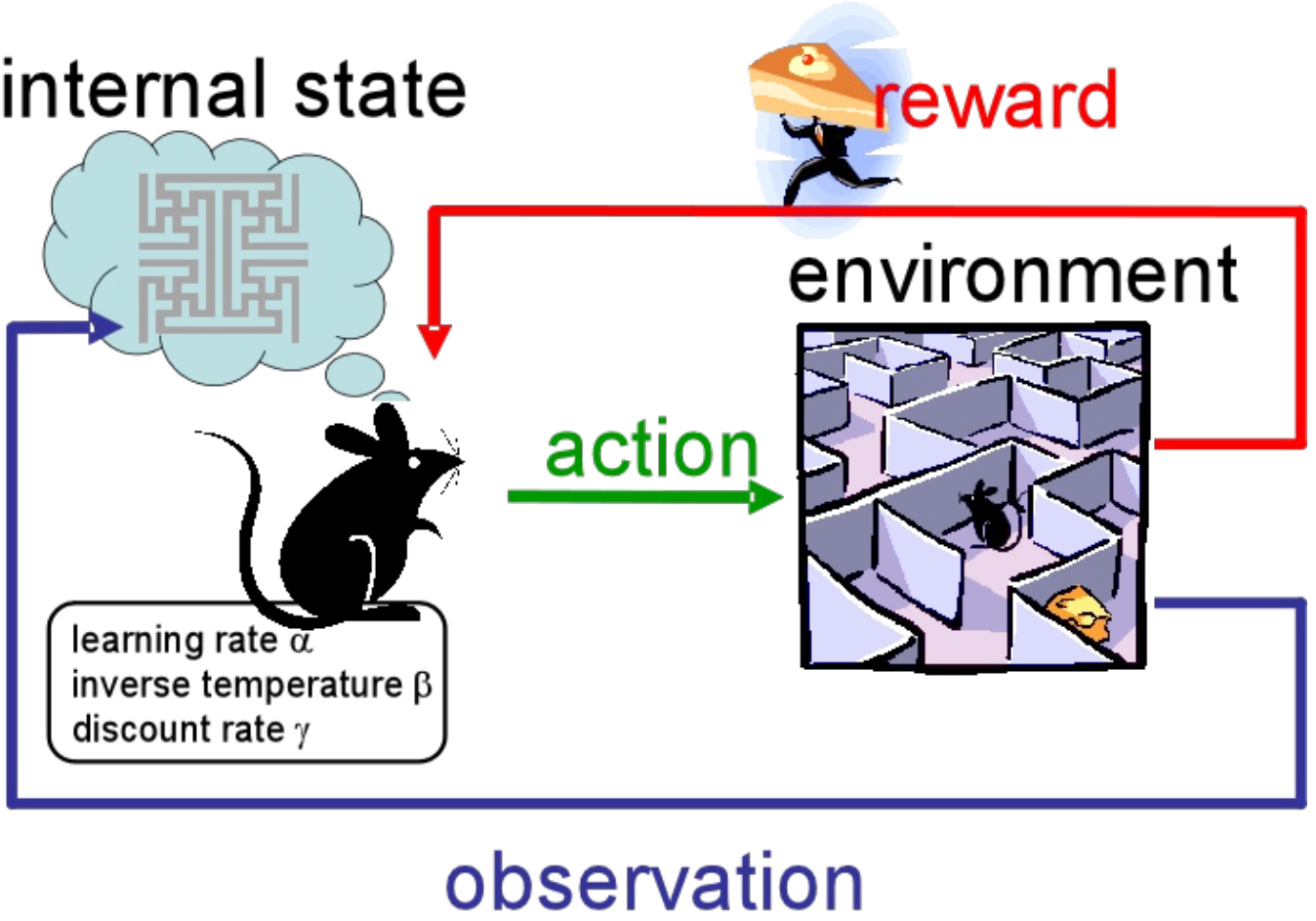
Reduce data from 2D to 1D



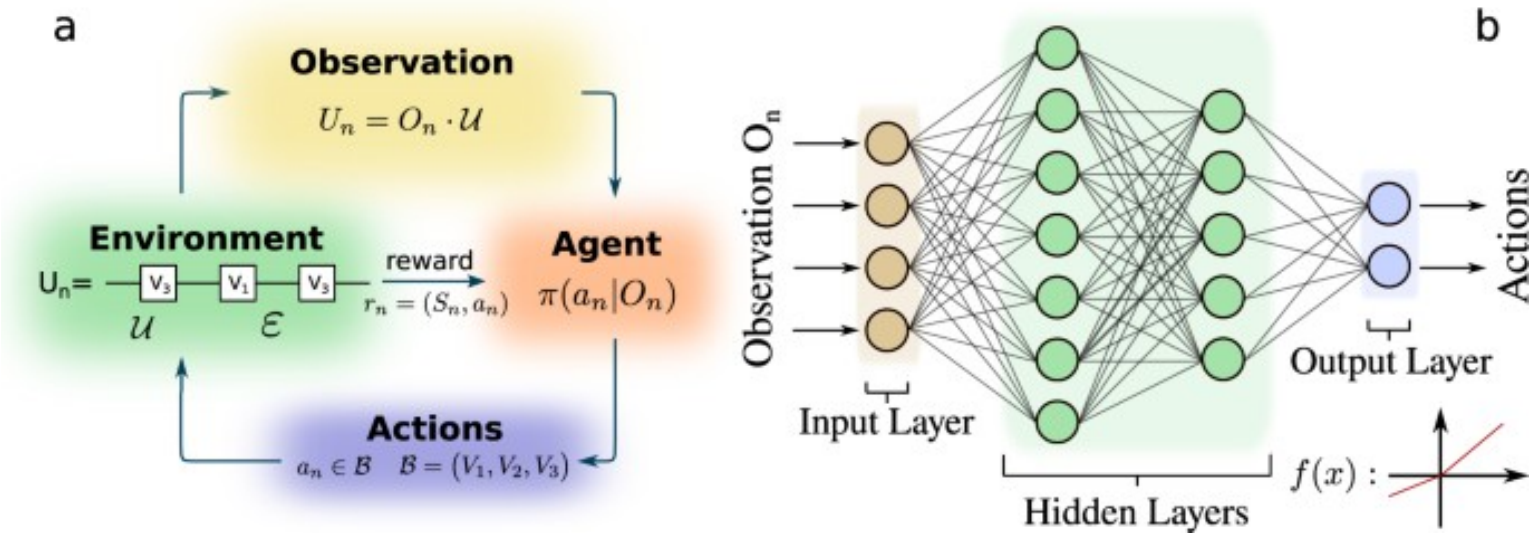
Reduce data from 3D to 2D

$z^{(i)}$ $x^{(i)} \in \mathbb{R}^2 \rightarrow z^{(i)} \in \mathbb{R}$

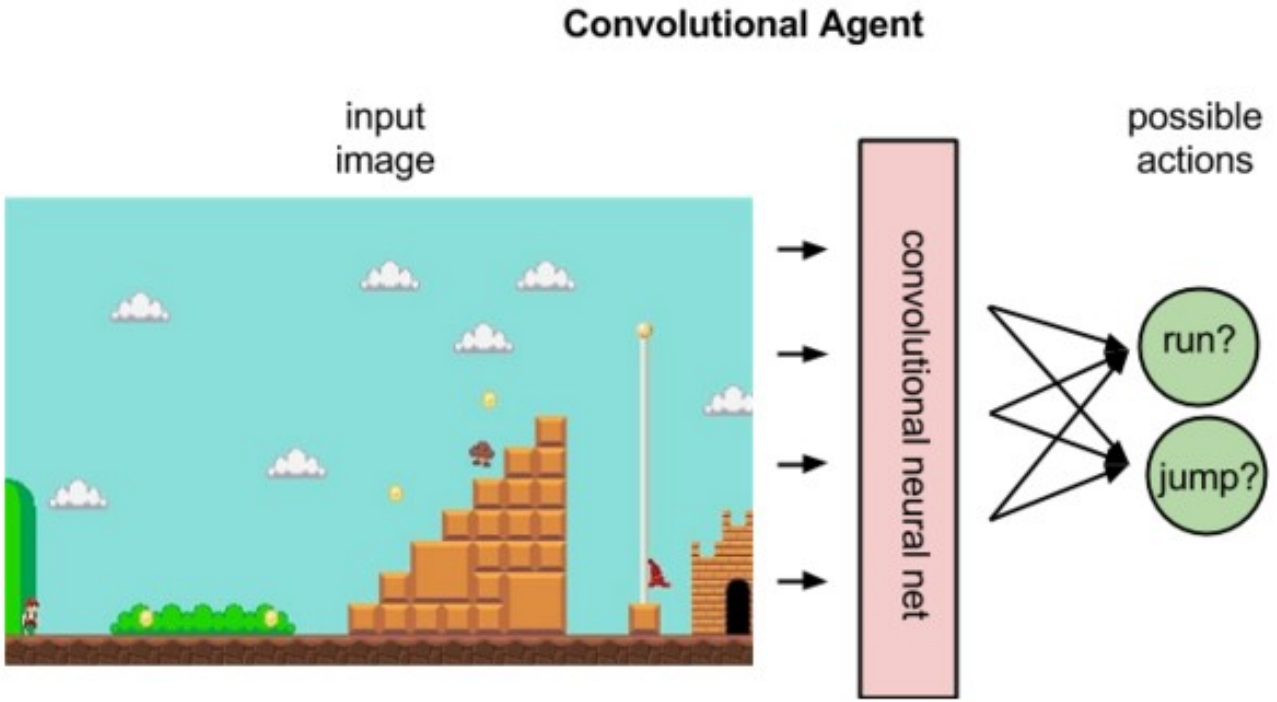
Aprendizado por reforço



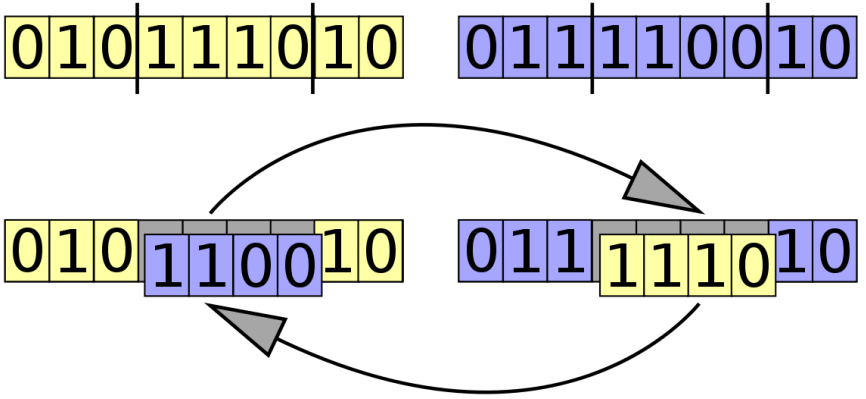
Deep Reinforcement



Deep Reinforcement



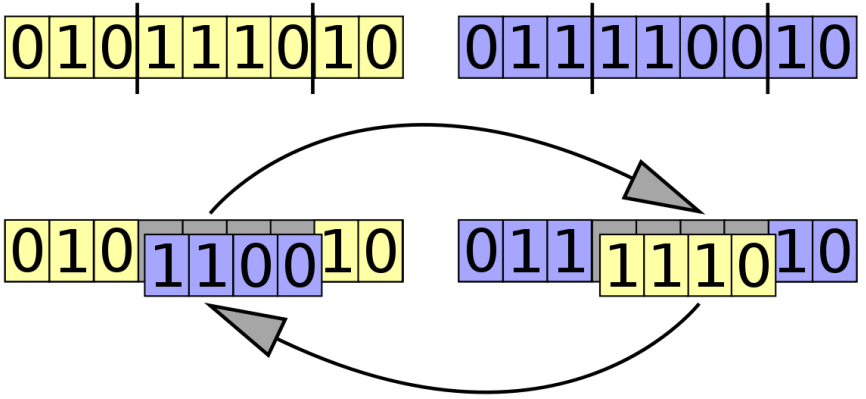
Algoritmo genético – Neuroevolução



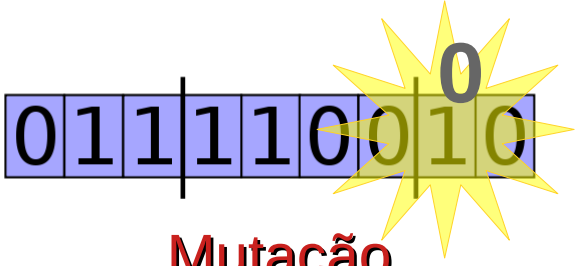
Cruzamento

Mutação

Algoritmo genético – Neuroevolução



Cruzamento

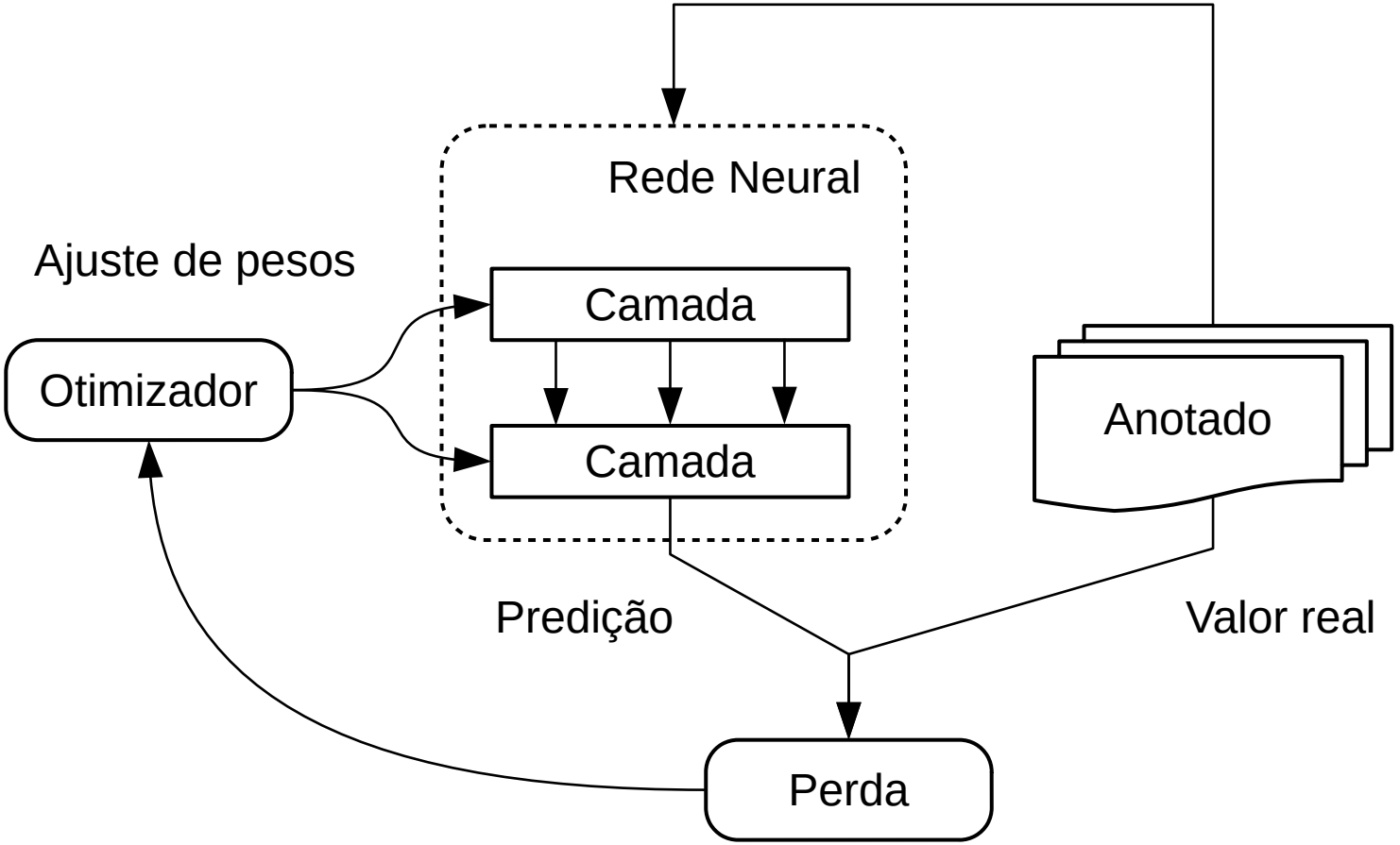


Mutação

Exercício

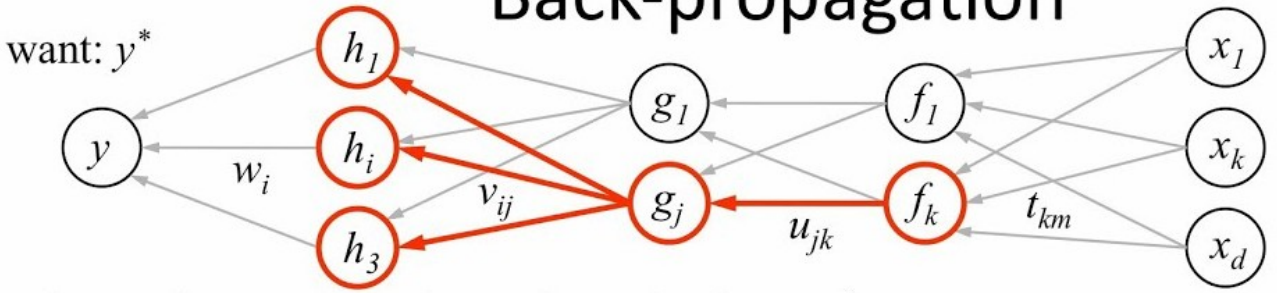
R: Como você classificaria esse tipo de aprendizado: por reforço, supervisionado ou não supervisionado?

Aprendizado supervisionado em RNA



Aprendizado supervisionado em RNA

Back-propagation



1. receive new observation $\boldsymbol{x} = [x_1 \ldots x_d]$ and target y^*
2. **feed forward:** for each unit g_j in each layer 1…L
   compute g_j based on units f_k from previous layer: $g_j = \sigma\left(u_{j0} + \sum_k u_{jk} f_k\right)$
3. get prediction y and error (y-y^*)
4. **back-propagate error:** for each unit g_j in each layer L…1

(a) compute error on g_j

$$\underbrace{\frac{\partial E}{\partial g_j}}_{\text{should } g_j \text{ be higher or lower?}} = \sum_i \underbrace{\sigma'(h_i) v_{ij}}_{\text{how } h_i \text{ will change as } g_j \text{ changes}} \underbrace{\frac{\partial E}{\partial h_i}}_{\text{was } h_i \text{ too high or too low?}}$$

(b) for each u_{jk} that affects g_j

(i) compute error on u_{jk}

$$\frac{\partial E}{\partial u_{jk}} = \underbrace{\frac{\partial E}{\partial g_j}}_{\text{do we want } g_j \text{ to be higher/lower}} \underbrace{\sigma'(g_j) f_k}_{\text{how } g_j \text{ will change if } u_{jk} \text{ is higher/lower}}$$

(ii) update the weight

$$u_{jk} \leftarrow u_{jk} - \eta \frac{\partial E}{\partial u_{jk}}$$

Treinamento de RNA's

- **Batch e Mini-batch**: Um batch (lote) é o conjunto completo de dados usado para atualizar os pesos da rede em uma iteração. Em mini-batch, apenas um subconjunto dos dados é usado, equilibrando eficiência computacional e estabilidade de gradiente.
- **Época**: Uma época é uma passagem completa pelos dados de treinamento, onde todos os exemplos são usados para ajustar o modelo.
- **Otimizadores**: Atualizam os pesos após cada exemplo (ou mini-batch), acelerando o treinamento em dados grandes.
- **Taxa de Aprendizado**: Controla o tamanho dos passos do otimizador ao ajustar os pesos da rede.

Treinamento de RNA's: otimizadores

- **SGD (Gradiente Estocástico)**: Simples e eficiente, adequado para problemas de aprendizado com alta dimensionalidade.
- **Adam**: Combina momentum e ajuste adaptativo de taxa de aprendizado, ideal para redes profundas e dados complexos.
- **RMSprop**: Ajusta a taxa de aprendizado por dimensão, eficiente para problemas com gradientes variáveis, como RNNs (Recursive Neural Network).
- **Adagrad**: Taxa de aprendizado decrescente, bom para dados esparsos, mas pode ser limitado em problemas maiores.

Treinamento de RNA's: funções de perda

- **MSE (Erro Quadrático Médio)**: Para regressão, penaliza grandes diferenças entre previsões e valores reais.
- **Cross-Entropy**: Usada em classificação, avalia discrepância entre distribuições preditas e reais.
- **MAE (Erro Absoluto Médio)**: Mais robusta a outliers em problemas de regressão.
- **Hinge Loss**: Para classificação binária, utilizada em SVMs (Support Vector Machine) e RNAs similares.

Variational Autoencoders (VAE's)

<u>Reconstruction Loss (Erro de Reconstrução)</u>: Mede a diferença entre a entrada e a saída reconstruída.

Geralmente baseada em MSE (Erro Quadrático Médio) para dados contínuos ou Cross-Entropy para dados categóricos.

<u>KL Divergence (Divergência de Kullback-Leibler)</u>: Regulariza a distribuição latente aproximada (posterior) para se alinhar com a distribuição latente prior (geralmente Gaussiana).

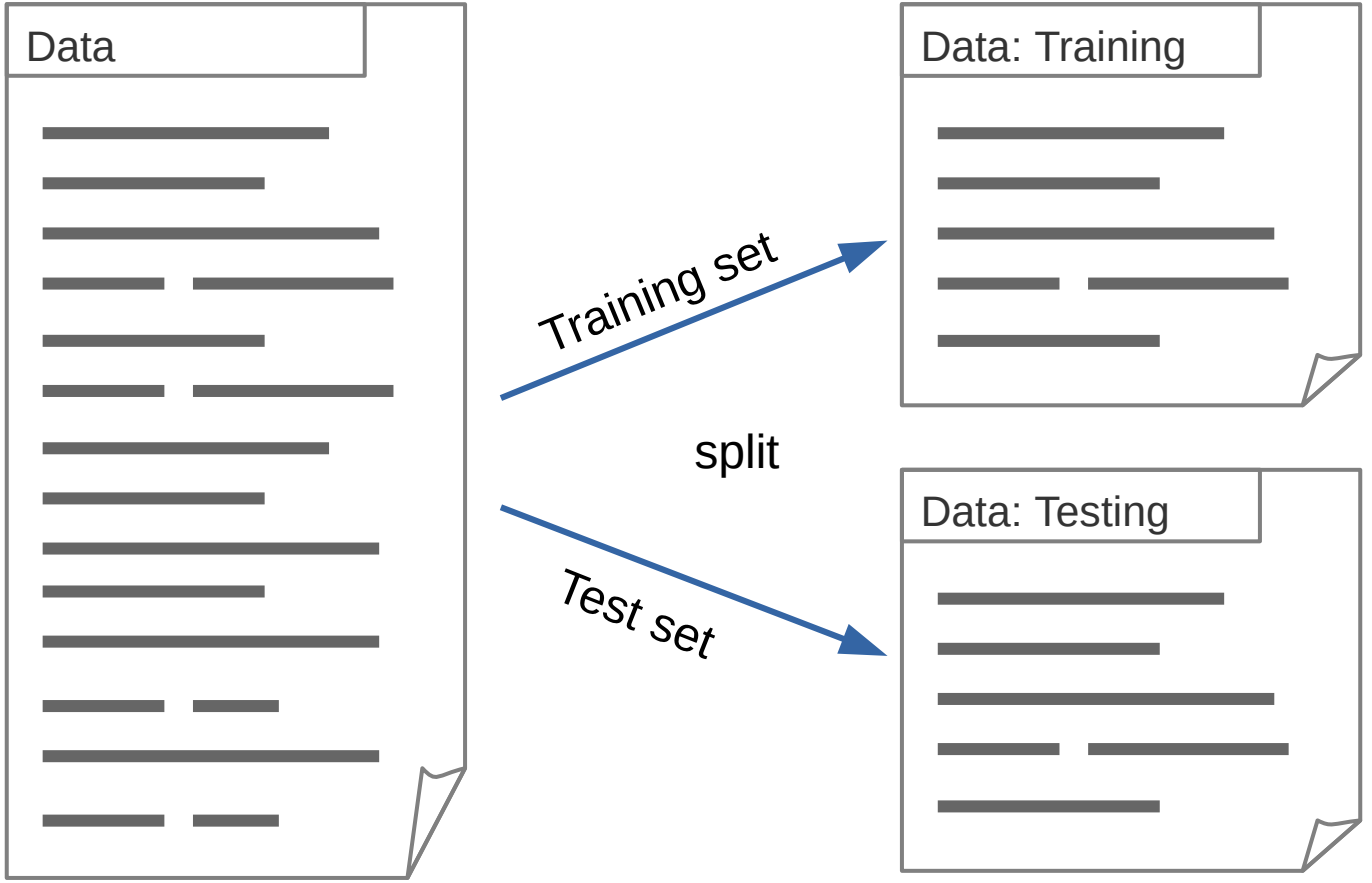
Situação: Fundamental para manter uma representação latente regularizada e contínua, crucial para a geração de novos dados.

A função de perda total é a soma ponderada dessas duas componentes:

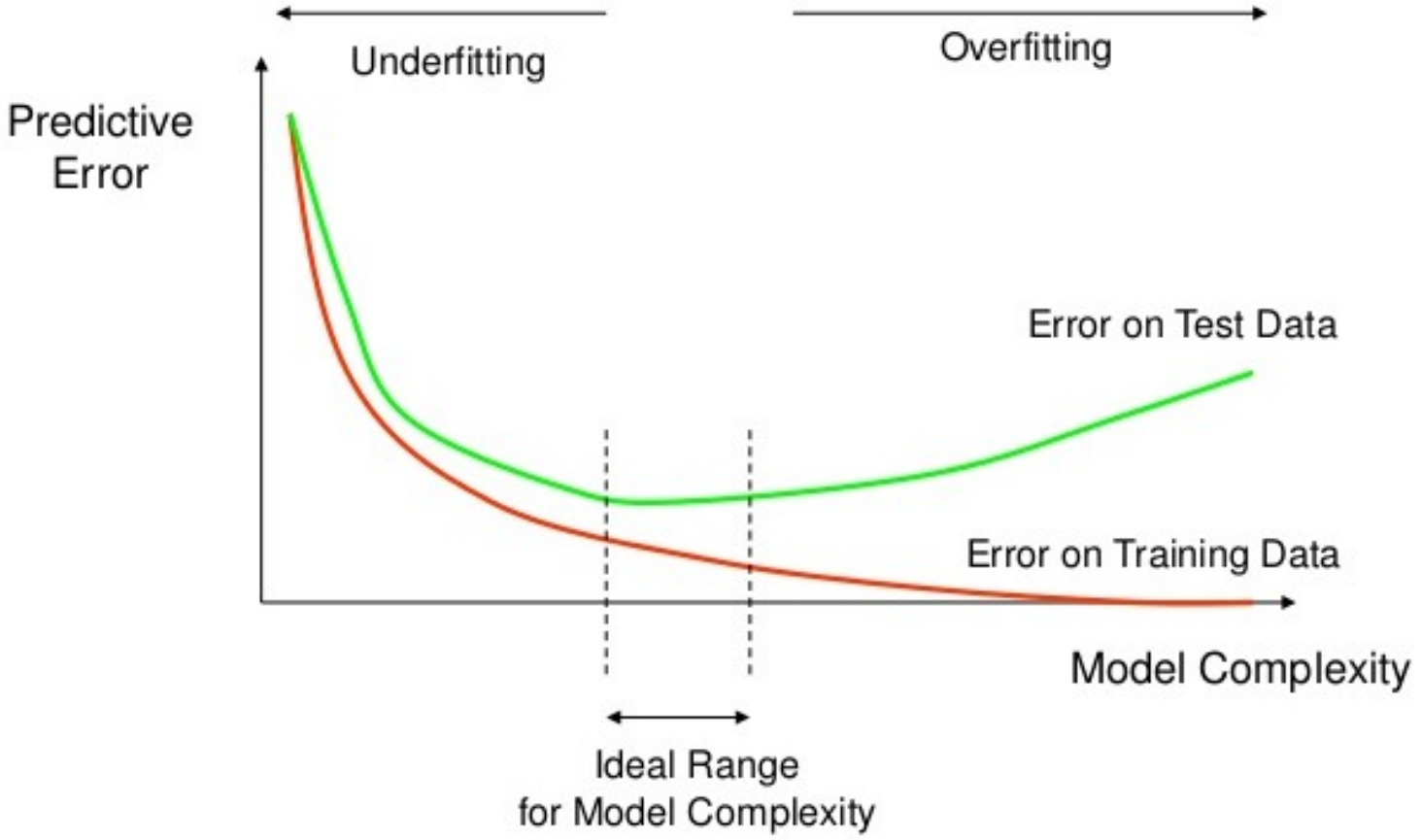
$$Loss = Reconstruction\ loss + \beta\, KL\ Divergence$$

O fator β ajusta a importância relativa da regularização.

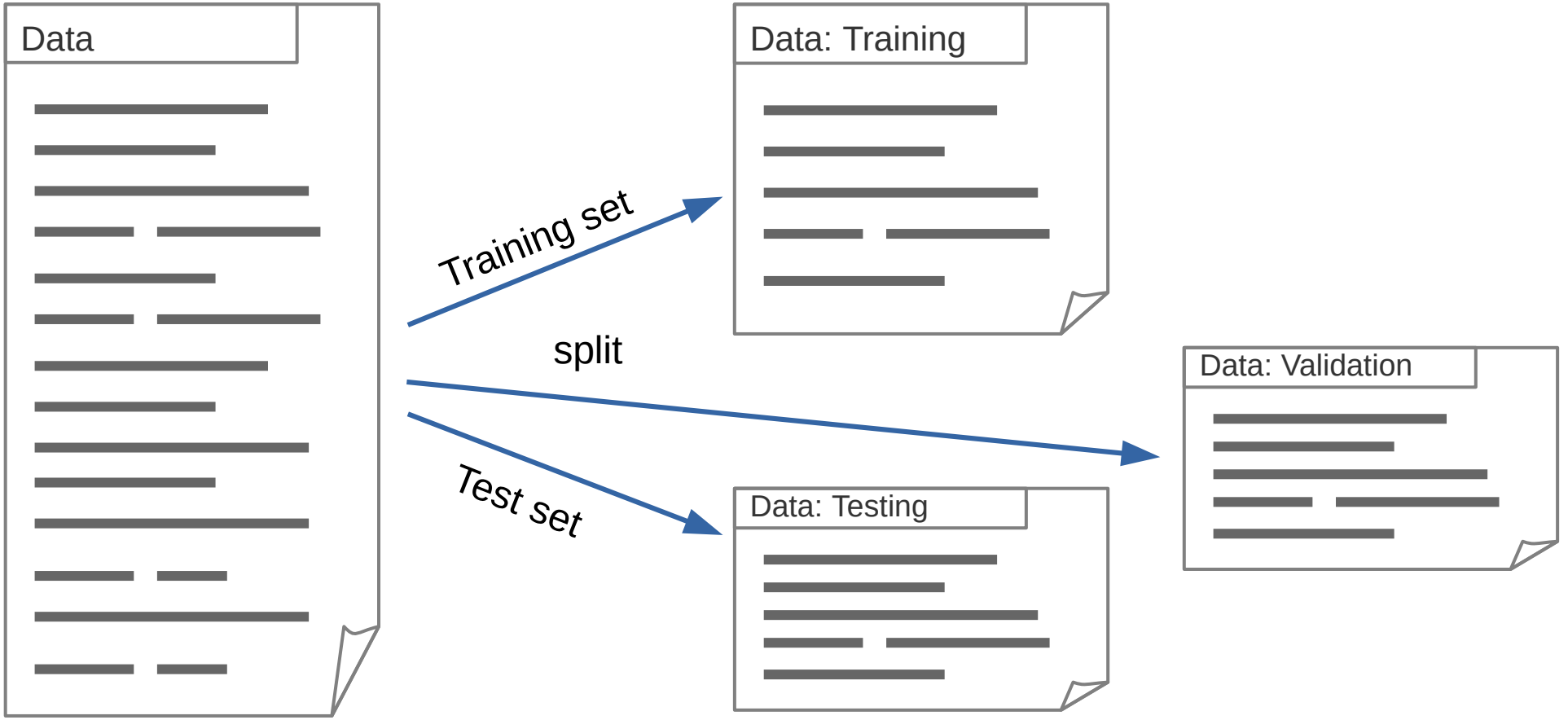
Conjunto de dados



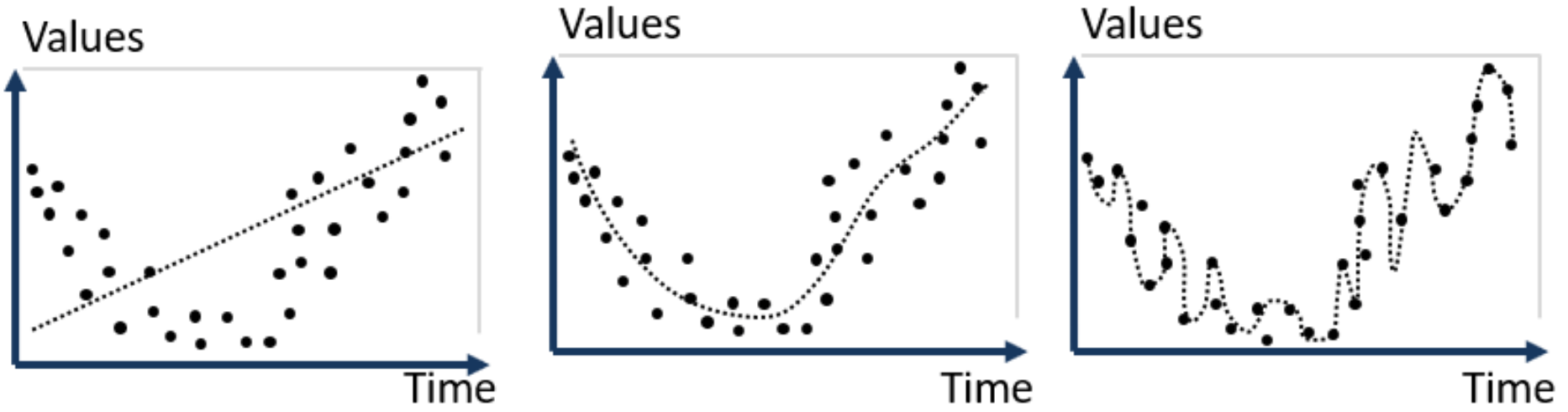
Erro de predição



Conjunto de dados



Erro de ajuste



Underfitted
(High bias error)

Good Fit/Robust
(Balance between bias and variance)

Overfitted
(High variance error)

Regularização: Objetivos

- Prevenir overfitting.
- Melhorar a robustez do modelo.
- Promover generalização para dados não vistos.
- Reduzir a complexidade ou dependência de partes específicas da rede.

Regularização

- A regularização em redes neurais artificiais é um conjunto de técnicas destinadas a melhorar a generalização do modelo, prevenindo ou reduzindo o overfitting.
- O objetivo principal da regularização é encontrar um equilíbrio entre o ajuste ao conjunto de treinamento e a capacidade de generalizar para dados não vistos. Isso é feito introduzindo restrições ou penalidades no treinamento da rede.

Principais Técnicas de Regularização

L1 e L2 Regularization

Least Absolute Shrinkage and Selection Operator ("Operador de Seleção e Redução Absoluta").

Descrição:

Consistem em adicionar penalidades aos pesos na função de perda.

L1 (lasso): Soma dos valores absolutos dos pesos.

$$Loss = Loss_{original} + \lambda \sum |w_i|$$

L2 (ridge): Soma dos quadrados dos pesos.

Literalmente, "crista", mas no contexto de aprendizado de máquina e estatística, é comumente traduzido como "Regularização de Ridge"

$$Loss = Loss_{original} + \lambda \sum w_i^2$$

CMCC
Centro de Matemática, Computação e Cognição

Principais Técnicas de Regularização

L1 e L2 Regularization

Objetivo:

L1: Incentivar sparsity, resultando em pesos zero, útil para seleção de características.

L2: Reduzir pesos grandes, incentivando distribuições mais uniformes.

Pontos Fortes:

L1: Gera redes mais compactas e simples.

L2: Suaviza os pesos, útil para estabilidade numérica.

Aplicações: Modelos lineares, CNNs e RNNs.

Principais Técnicas de Regularização

Elastic Net

Descrição:

Combinação de L1 e L2 regularization.

A função de perda inclui termos de ambas as regularizações:

$$Loss = Loss_{original} + \lambda_1 \sum |w_i| + \lambda_2 \sum w_i^2$$

Objetivo:

Combinar sparsity (L1) com suavização (L2).

Pontos Fortes:

Eficaz em redes onde sparsity e estabilidade são importantes.

Aplicações: Modelos preditivos complexos.

Principais Técnicas de Regularização

Dropout

Descrição:

Durante o treinamento, desativa aleatoriamente um percentual de neurônios em cada camada, forçando a rede a não depender de combinações específicas de neurônios.

Objetivo:

Reduzir a coadaptação de neurônios, promovendo redes mais robustas.

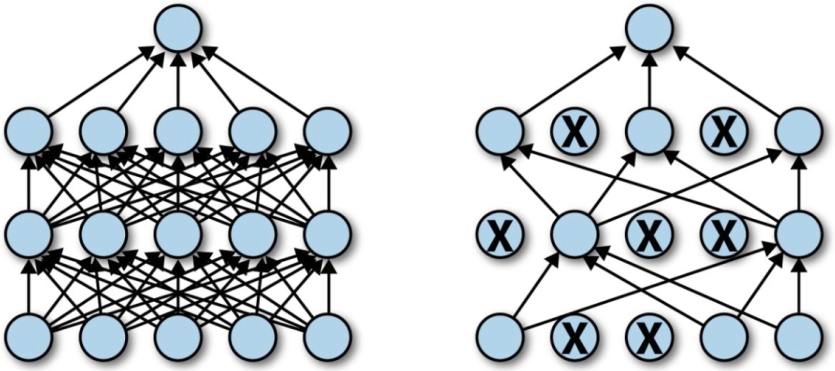
Pontos Fortes:

Simples e eficaz, reduz significativamente o overfitting.

Pontos Fracos:

Pode dificultar a convergência inicial.

Aplicações: Redes profundas, como CNNs e MLPs.



Principais Técnicas de Regularização

Early Stopping

Descrição:

Interrompe o treinamento quando o desempenho no conjunto de validação começa a piorar.

Objetivo:

Evitar o overfitting ao limitar o número de épocas de treinamento.

Pontos Fortes:

Simples de implementar, eficaz para grandes datasets.

Pontos Fracos:

Depende fortemente de um conjunto de validação bem separado.

Aplicações: Qualquer modelo treinado iterativamente.

Principais Técnicas de Regularização

Data Augmentation

Descrição:

Aumenta artificialmente o conjunto de dados aplicando transformações (e.g., rotações, espelhamentos, adição de ruído).

Objetivo:

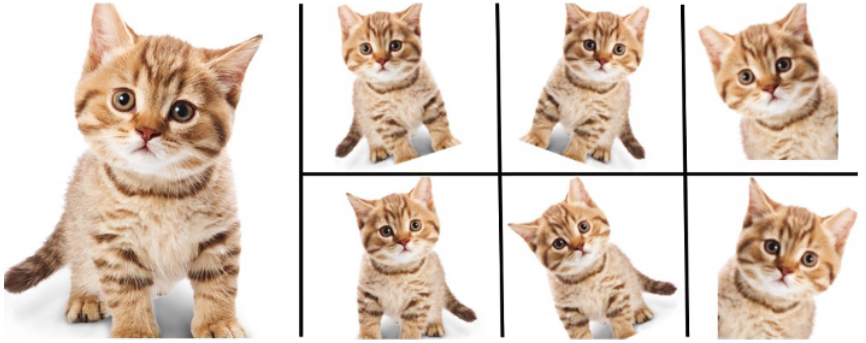
Tornar o modelo mais robusto a variações e melhorar a generalização.

Pontos Fortes:

Eficaz em conjuntos de dados pequenos.

Pontos Fracos:

Exige criatividade para criar variações realistas.



Aplicações: Principalmente em CNNs para visão computacional.

Principais Técnicas de Regularização

Batch Normalization

Descrição:

Normaliza as ativações de cada camada em mini-batches durante o treinamento, reduzindo a covariância interna.

Objetivo:

Acelerar o treinamento e reduzir a sensibilidade a inicializações de pesos.

Pontos Fortes:

Funciona como regularizador adicional.

Pontos Fracos:

Menos eficaz em redes muito pequenas.

Aplicações: Redes profundas.

Principais Técnicas de Regularização

<u>Weight Constraint</u>

Descrição:

Impõe limites nos valores dos pesos (e.g., norm clipping).

Objetivo:

Evitar que os pesos cresçam descontroladamente.

Pontos Fortes:

Simples de implementar.

Pontos Fracos:

Menos versátil do que L2 regularization.

Aplicações: RNNs e redes profundas.

Principais Técnicas de Regularização

Noise Injection

Descrição:

Adiciona ruído aos dados de entrada, pesos ou ativações durante o treinamento.

Objetivo:

Simular a presença de incertezas, tornando a rede mais robusta.

Pontos Fortes:

Ajuda a reduzir a sensibilidade a pequenas variações.

Pontos Fracos:

Requer ajuste cuidadoso da magnitude do ruído.

Aplicações: Redes para aprendizado robusto.

Principais Técnicas de Regularização

Adversarial Training

Descrição:

Introduz exemplos adversariais no treinamento para tornar a rede mais robusta a ataques ou ruídos.

Objetivo:

Aprimorar a capacidade da rede de lidar com entradas adversárias.

Pontos Fortes:

Aumenta a robustez geral do modelo.

Pontos Fracos:

Requer maior poder computacional.

Aplicações: Segurança em redes neurais.

Regularização: Escolhas

- Redes profundas:
  - Combine Dropout, Batch Normalization e Data Augmentation.
- Redes pequenas:
  - Use L1/L2 Regularization e Early Stopping.
- Conjuntos de dados pequenos:
  - Aposte em Data Augmentation e Noise Injection.
- Aplicações robustas:
  - Invista em Adversarial Training.

Bibliotecas em Python

Keras

PyTorch